

Esame Programmazione di Sistema / API Programming

Data: 2024-01-22

QUESTIONS

Domanda 1

Sia data la struttura `LinkedList<T>` definita come:

```
1 pub struct LinkedList<'a, T> {  
2     pub val: Option<T>,  
3     pub next: Option<&'a Box<LinkedList<'a, T>>>,  
4 }
```

Si definisca l'occupazione di memoria di un elemento della lista e si indichi come sia possibile definire il fine lista.

Domanda 2

Si definisca un esempio in cui, data la necessita' di creare N thread, si possano evitare *race conditions* nel momento in cui i thread debbano accedere in scrittura alla stessa risorsa. Si distingua il caso in cui tale risorsa sia uno scalare e quella in cui sia una struttura piu' articolata.

Domanda 3

Si usi un esempio concreto per dimostrare la capacita' di introdurre modularita' attraverso i Tratt.

Domanda 4

La struttura `MultiChannel` implementa il concetto di canale con molti mittenti e molti ricevitori. I messaggi inviati a questo tipo di canale sono composti da singoli byte che vengono recapitati a tutti i ricevitori attualmente collegati.

Riferimenti a tipi:

```
1 use std::result::Result;  
2 use std::sync::mpsc::{Receiver, SendError};
```

Metodi:

- `new() -> Self`
crea un nuovo canale senza alcun ricevitore collegato.
- `subscribe(&self) -> Receiver<u8>`
collega un nuovo ricevitore al canale: da quando questo metodo viene invocato, gli eventuali byte inviati al canale saranno recapitati al ricevitore. Se il ricevitore viene eliminato, il canale continuerà a funzionare inviando i propri dati ai ricevitori restanti (se presenti), altrimenti ritornerà un errore.

- `send(&self, data: u8) -> Result<(), SendError<u8>`
invia a tutti i sottoscrittori un byte; se non c'è alcun sottoscrittore, notifica l'errore indicando il byte che non è stato trasmesso.

SOLUTIONS

Domanda 1

L'occupazione di memoria della struttura `LinkedList<'a, T>` è la somma della memoria occupata dai suoi campi, tenendo conto dei requisiti di allineamento (padding) imposti dall'architettura e dal compilatore. I due campi sono:

- `val: Option<T>`
- `next: Option<&'a Box<LinkedList<'a, T>>`

Il campo `next` è un'opzione contenente un riferimento (puntatore) a un `Box`. I riferimenti non possono mai essere nulli. Grazie alla *Null Pointer Optimization (NPO)* effettuata dal compilatore Rust, `Option<T>` occupa la stessa memoria del riferimento `&T` stesso, utilizzando il puntatore nullo per rappresentare la variante `None`. Di conseguenza, il campo `next` occupa esattamente la dimensione di un puntatore di sistema (8 byte su architetture a 64 bit), che ha un allineamento di 8 byte.

L'occupazione del campo `val: Option<T>` dipende invece dalla dimensione del tipo `T`. Nel caso generale, un `Option<T>` occupa `size_of::<T>()` più 1 byte di discriminante, a meno che il tipo `T` non sia soggetto a ottimizzazioni specifiche (come accade per i puntatori, `NonZeroU32`, ecc.). Se assumiamo un'architettura a 64 bit e, ad esempio, `T = i32`:

- `val: Option<i32>` occupa 4 byte per l'`i32` e 1 byte per il discriminante dell'enumerazione. Tuttavia, l'allineamento richiesto dall'`i32` è di 4 byte, per cui la dimensione totale di `val` sarebbe arrotondata a 8 byte.
- `next: Option<&Box<...>` occupa 8 byte e richiede allineamento a 8 byte.

Di conseguenza, su un sistema a 64 bit con `T = i32`, la dimensione totale della struttura sarebbe di 16 byte (8 byte per `val` e il suo padding, seguiti da 8 byte per il riferimento contenuto in `next`). L'allineamento della struttura nel suo complesso sarebbe di 8 byte. Nota: nel caso generale, l'occupazione dipende dalla dimensione e dall'allineamento di `T`.

Il "fine lista" viene definito impostando il campo `next` a `None`. Essendo una lista implementata attraverso ricorsione sul riferimento (ed allocazione sul mucchio `Box`), quando un nodo non ha alcun nodo successivo da riferire, memorizzerà `None` nella variante di `Option`, interrompendo la catena.

Domanda 2

Per evitare le *race conditions* in presenza di più thread che devono accedere in scrittura a una risorsa condivisa, in Rust ci si avvale solitamente di costrutti specifici. Non basta la concorrenza "cruda" del sistema operativo, ma Rust impone il concetto di sicurezza a compile-time (grazie a `Send` e `Sync` e ai controlli del borrow checker).

1. Risorsa scalare (es. `usize`) Quando la risorsa da proteggere è un tipo primitivo che può essere letto o scritto con operazioni indivisibili dal processore (es. flag booleani o interi contatori), Rust fornisce i tipi atomici definiti in `std::sync::atomic` (come `AtomicUsize`, `AtomicBool`). Essi offrono la mutabilità interna e l'accesso concorrente senza necessità di bloccare in *lock* gli altri thread.

```

1 use std::sync::atomic::{AtomicUsize, Ordering};
2 use std::sync::Arc;
3 use std::thread;
4
5 fn main() {
6     let counter = Arc::new(AtomicUsize::new(0));
7     let mut handles = vec![];
8
9     for _ in 0..10 {
10         let counter_clone = Arc::clone(&counter);
11         let handle = thread::spawn(move || {
12             // Operazione di incremento atomico, non occorre lock
13             counter_clone.fetch_add(1, Ordering::Relaxed);
14         });
15         handles.push(handle);
16     }
17
18     for handle in handles {
19         handle.join().unwrap();
20     }
21     println!("Counter: {}", counter.load(Ordering::SeqCst));
22 }

```

2. Struttura piu' articolata Nel caso in cui la risorsa condivisa sia complessa (es. `Vec`, `HashMap` o una struct), si deve utilizzare un blocco mutuo-esclusivo (`Mutex` o `RwLock`) all'interno di uno smart pointer `Arc` (Atomic Reference Counter). L'`Arc` gestisce la vita del dato tra piu' thread, ed il `Mutex` regola gli accessi concorrenti garantendo la mutabilita' interna.

```

1 use std::sync::{Arc, Mutex};
2 use std::thread;
3
4 fn main() {
5     let data = Arc::new(Mutex::new(vec![]));
6     let mut handles = vec![];
7
8     for i in 0..10 {
9         let data_clone = Arc::clone(&data);
10        let handle = thread::spawn(move || {
11            // Ottengo il lock esclusivo
12            let mut vec = data_clone.lock().unwrap();
13            vec.push(i);
14        });
15        handles.push(handle);
16    }
17
18    for handle in handles {
19        handle.join().unwrap();
20    }
21
22    // Il thread principale legge il risultato finale
23    let final_vec = data.lock().unwrap();
24    println!("Dati: {:?}", *final_vec);
25 }

```

Domanda 3

In Rust, i tratti definiscono un set di comportamenti condivisi che diversi tipi possono implementare. Svolgono il ruolo delle interfacce nella programmazione a oggetti, introducendo

modularità nel codice, ad esempio separando la definizione di un'operazione dalle sue implementazioni concrete.

Supponiamo di creare un sistema per inviare report e notifiche tramite vari mezzi. Definiamo un tratto `Notifier` con un metodo `send_message`. Vari tipi (`EmailNotifier`, `SmsNotifier`) lo implementeranno diversamente. Le funzioni che dipendono dal meccanismo di notifica possono così operare in maniera generica, appoggiandosi al tratto senza conoscere il tipo esatto (*polimorfismo a tempo di compilazione o di esecuzione*).

```
1 // 1. Definizione del tratto
2 pub trait Notifier {
3     fn send_message(&self, message: &str);
4 }
5
6 // 2. Tipi che implementano il tratto
7 pub struct EmailNotifier { pub address: String }
8
9 impl Notifier for EmailNotifier {
10     fn send_message(&self, message: &str) {
11         println!("Invio email a {}: {}", self.address, message);
12     }
13 }
14
15 pub struct SmsNotifier { pub phone_number: String }
16
17 impl Notifier for SmsNotifier {
18     fn send_message(&self, message: &str) {
19         println!("Invio SMS a {}: {}", self.phone_number, message);
20     }
21 }
22
23 // 3. Uso modulare: genericità a compile-time (monomorfizzazione)
24 fn alert_user<T: Notifier>(notifier: &T, msg: &str) {
25     notifier.send_message(msg);
26 }
27
28 // Uso modulare alternativo: genericità a run-time (dynamic dispatch)
29 fn notify_all(notifiers: Vec<Box<dyn Notifier>>, msg: &str) {
30     for n in notifiers {
31         n.send_message(msg);
32     }
33 }
34
35 fn main() {
36     let email = EmailNotifier { address: "utente@example.com".to_string() };
37     let sms = SmsNotifier { phone_number: "+391234567".to_string() };
38
39     alert_user(&email, "Attenzione: Login effettuato.");
40     alert_user(&sms, "Attenzione: Login effettuato.");
41
42     let mixed_notifiers: Vec<Box<dyn Notifier>> = vec![
43         Box::new(EmailNotifier { address: "admin@example.com".to_string() }),
44         Box::new(SmsNotifier { phone_number: "+399876543".to_string() })
45     ];
46     notify_all(mixed_notifiers, "Sistema aggiornato!");
47 }
```

La modularita' e' evidente: e' possibile aggiungere un nuovo mezzo di notifica (es. `PushNotifier`) e la logica core nelle funzioni `alert_user` e `notify_all` non necessitera' di alcuna modifica, semplicemente accettera' nuovi tipi che implementano `Notifier`.

Domanda 4

L'implementazione per il `MultiChannel` utilizza `mpsc::channel` di base per inviare i messaggi ad ogni "sottoscrittore". La struttura ospitera' una collezione protetta di `Sender<u8>`. Quando `send` viene richiamato, itera sulla lista dei sender inviando il byte. Se un `Sender` ritorna un `SendError`, significa che il corrispondente ricevitore e' stato scartato (droppato), quindi approfittiamo per pulire il vettore in modo da non allocare risorse inutilmente. Se al termine dell'invio (o prima di inviare) si nota che nessun trasmettitore e' rimasto iscritto (ovvero il vettore e' vuoto), ritorneremo un `Err(SendError(data))`.

```
1 use std::sync::mpsc::{channel, Receiver, Sender, SendError};
2 use std::sync::Mutex;
3
4 pub struct MultiChannel {
5     senders: Mutex<Vec<Sender<u8>>>,
6 }
7
8 impl MultiChannel {
9     pub fn new() -> Self {
10         Self {
11             senders: Mutex::new(Vec::new()),
12         }
13     }
14
15     pub fn subscribe(&self) -> Receiver<u8> {
16         // Creiamo un nuovo canale per questo particolare
17         sottoscrittore
18         let (tx, rx) = channel();
19
20         // Proteggiamo l'accesso al vettore con il Mutex
21         let mut guard = self.senders.lock().unwrap();
22         guard.push(tx);
23
24         // Ritorniamo la parte ricevente al chiamante
25         rx
26     }
27
28     pub fn send(&self, data: u8) -> Result<(), SendError<u8>> {
29         let mut guard = self.senders.lock().unwrap();
30
31         // Inviando il byte a tutti i sender attualmente presenti.
32         // Utilizziamo 'retain' per iterare sul vettore: manteniamo
33         solo
34         // i Sender per i quali 'send(data)' ha successo (Ok). I
35         riceventi
36         // deallocati causeranno un Err, quindi il loro sender verra'
37         rimosso.
38         guard.retain(|tx| tx.send(data).is_ok());
39
40         // Se dopo la pulizia o prima dell'invio il vettore e' vuoto,
41         // significa che nessun ricevente era pronto a leggere il byte
42         // (zero sottoscrittori). Si ritorna SendError contenente il
43         dato
```

```
39         // non inviato a nessuno.
40         if guard.is_empty() {
41             Err(SendError(data))
42         } else {
43             Ok(())
44         }
45     }
46 }
```